

PROJETO FINAL

Reconhecimento de Atividades Humanas com TinyML

ESP32-S3 + MPU6050 + TensorFlow Lite Micro

UC IA Embarcada e Modelos Compactos

6 atividades

modelo INT8

Wokwi validado

TFLM

Gabriel Monteiro de Souza
Renan Cardoso dos Santos
Rodrigo Teles Dondé

81,13%
accuracy INT8

4.240 B
modelo INT8

21
testes aprovados

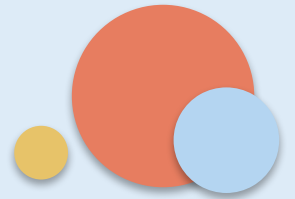
6/6
REPLAY no Wokwi

Proposta do projeto

Classificar atividades humanas localmente, com um modelo compacto, quantizado e executando no ESP32-S3.

Problema e objetivo

Por que usar IA embarcada para reconhecer atividades humanas?



Problema

Seis atividades precisam ser diferenciadas a partir de sinais de aceleração e rotação. Classes diferentes podem compartilhar amplitudes semelhantes.

Por que IA?

O modelo aprende a combinação entre os seis eixos e permite inferência local, com baixa latência e sem depender da nuvem.

WALKING
caminhar

UPSTAIRS
subir

DOWNSTAIRS
descer

SITTING
sentar

STANDING
em pé

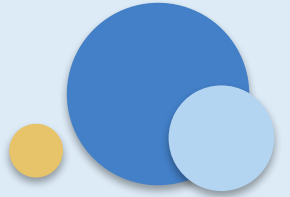
LAYING
deitado

Objetivo do protótipo

Dados inerciais → features → modelo → quantização → ESP32-S3 → classe + confiança.

Dados e hardware

Do UCI HAR ao ESP32-S3 com MPU6050



Dataset - UCI HAR

- Smartphone fixado na cintura
- 50 Hz
- Janela de 128 amostras = 2,56 s
- 6 atividades
- Aceleração + giroscópio em 3 eixos

Hardware embarcado

- ESP32-S3
- MPU6050 simulado no Wokwi
- I2C nos GPIO 8/9
- TensorFlow Lite Micro
- Inferência executada localmente

Divisão sem vazamento

Treino: 5.551 janelas
Validação: 1.801 janelas
Teste: 2.947 janelas

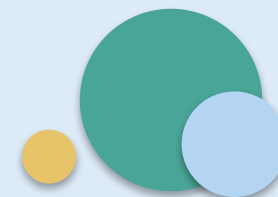
Os grupos de treino, validação e teste usam voluntários diferentes.

Ponto metodológico importante

A accuracy de 81,13% refere-se ao teste UCI HAR. O modo LIVE comprova a execução da pipeline com o MPU6050, mas não substitui uma validação estatística com dados próprios.

Pipeline de ponta a ponta

A mesma lógica é reproduzida no treinamento e no firmware



36 features por janela

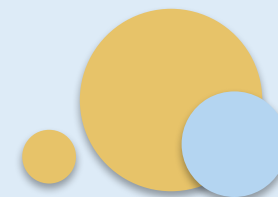
Para cada canal: média, desvio-padrão, RMS, mínimo, máximo e energia.

Equivalência validada

21 testes aprovados para features, quantização, golden vectors, janelas e artefatos Python/C.

Modelo e compressão

Rede pequena, quantização agressiva e impacto mínimo no desempenho



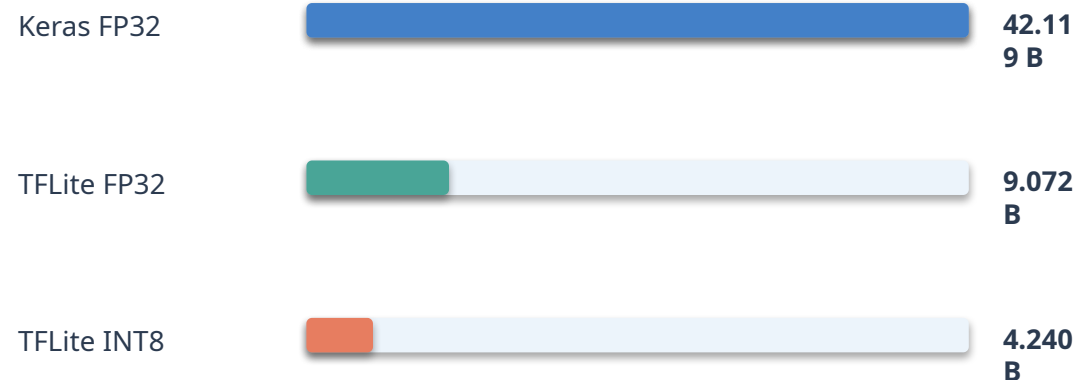
Arquitetura da MLP



1.814 parâmetros

FC + Softmax

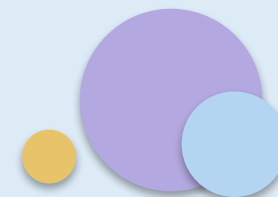
Tamanho dos artefatos



TFLite FP32 → INT8: redução de 53,26%

Resultados medidos

Teste oficial com 2.947 janelas de pessoas não vistas



81,13%

accuracy INT8

0,8070

macro F1

-0,136 pp

queda vs FP32

98,85%

concordância FP32/INT8

Onde o modelo confunde mais

- Caminhar × subir × descer escadas
- Sentado × em pé

LAYING foi a classe mais estável. O comportamento é coerente com um modelo compacto baseado em 36 estatísticas.

Impacto da quantização

Keras FP32: 81,27%
TFLite FP32: 81,27%
TFLite INT8: 81,13%

A queda de apenas 0,136 ponto percentual mostra que o modelo se adapta bem ao deploy INT8.

Deploy e demonstração no Wokwi

Do REPLAY reprodutível ao modo LIVE com sensor

REPLAY

Seis janelas reais do teste passam pela pipeline completa no ESP32-S3.
Resultado: 6/6 PASS.

LIVE

O MPU6050 simulado é lido por I2C. Em repouso, o sistema previu LAYING com alta confiança.

```
HAR: firmware ESP32-S3 / ANSI C
modo=REPLAY windows=6
replay WALKING_DOWNSTAIRS PASS
replay summary: passed=6 total=6 status=PASS
```

4.240 B
modelo

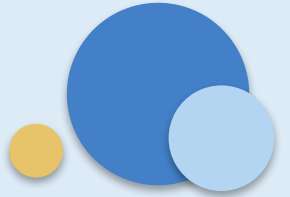
32 KiB
arena

≈ 3,1 ms
Invoke no Wokwi

6/6 PASS = paridade numérica com o desktop nas janelas selecionadas. Não significa 100% de accuracy.

Conclusões e próximos passos

O que foi demonstrado e o que ainda precisa ser validado



O que o projeto comprova

- Pipeline completa de TinyML
- Modelo INT8 pequeno e funcional
- 81,13% de accuracy no UCI HAR
- 21 testes aprovados
- REPLAY 6/6 PASS no Wokwi
- Modo LIVE executando com MPU6050

Limitação principal

Existe diferença de domínio entre o smartphone do UCI HAR e o MPU6050. Por isso, a accuracy real do LIVE ainda não foi medida.

Próximo passo técnico

Coletar dados rotulados com o próprio MPU6050, recalibrar a normalização e reavaliar o modelo com pessoas independentes.

github.com/rodrigodondesenai/projeto-final-tinyml-har